

Algorithms for Query Processing and Optimization

Chapter 2

Outline

- Introduction
- Processing a query
- SQL queries and relational algebra
- Implementing basic query operations
- Heuristics-based query optimization
- Selectivity and Cost Estimates in Query Optimization
- Semantic Query Optimization

INTRODUCTION

INTRODUCTION

- **Query Processing:** The process by which the query results are retrieved from a high-level query such as SQL or OQL.
- **Query Optimization:** The process of choosing a suitable execution strategy for retrieving results of query from database files for processing a query is known as Query Optimization.

Two main Techniques for Query Optimization

- Heuristic Rules

Rules for ordering the operations in query optimization.

- Systematical estimation

It estimates cost of different execution strategies and chooses the execution plan with lowest execution cost

QUERY DATA STRUCTURE

- Before optimizing the query it is represented in an internal or intermediate form.

Two internal representations of a query:

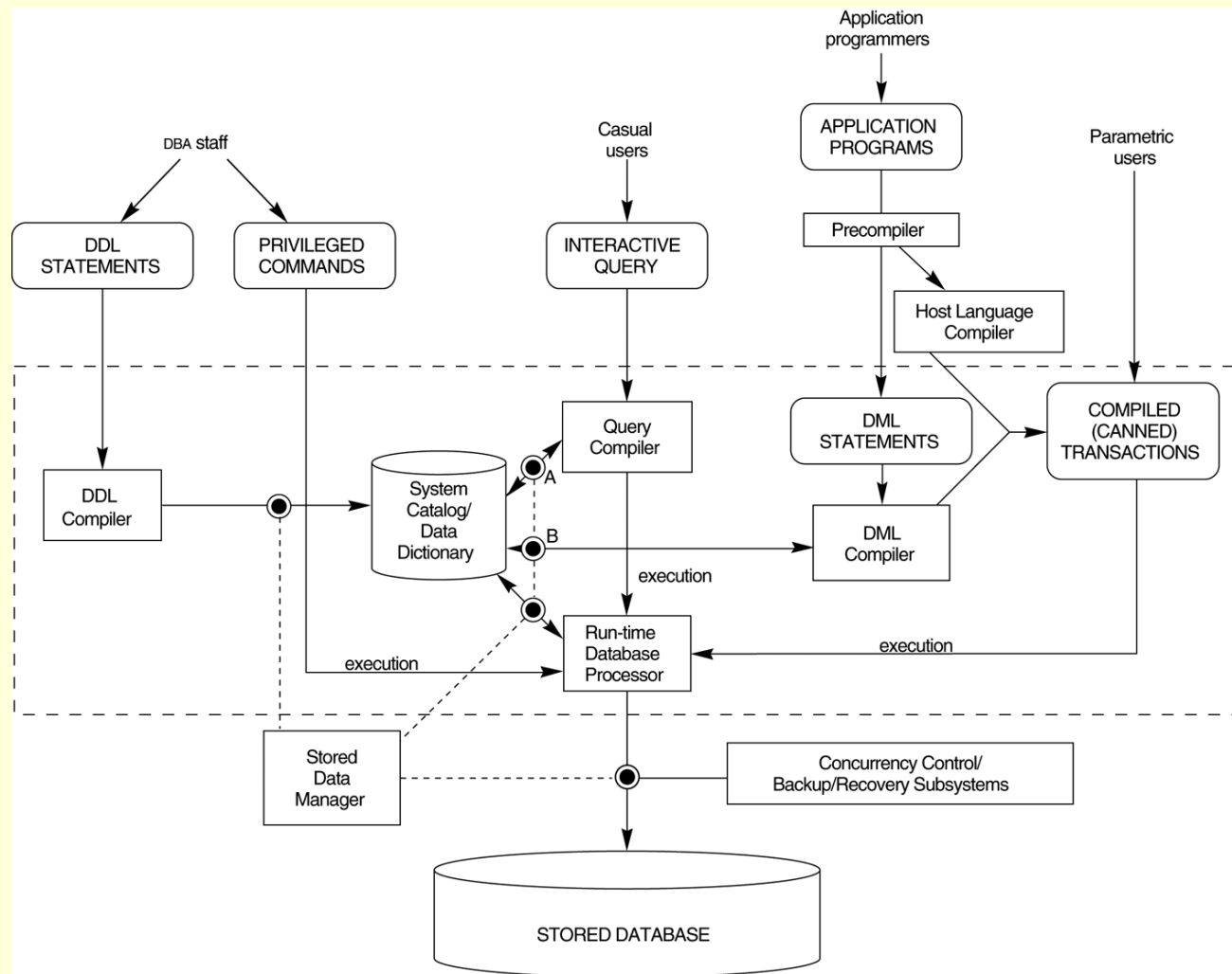
It is created using two data structures

- **Query tree:** A tree data structure that corresponds to a relational algebra expression. It represents the input relations of the query as *leaf nodes* of the tree, and represents the relational algebra operations as *internal nodes*.
- **Query graph:** A graph data structure that corresponds to a relational calculus expression. It does **not** indicate an order on which operations to perform first. There is only a **single** graph corresponding to each query.

QUERY PROCESSING

Background Review

- DDL compiler
- DML compiler
- Runtime database processor
- System catalog



Processing a Query

- Tasks in processing a high-level query
 1. Scanner scans the query and identifies the language tokens
 2. Parser checks syntax of the query
 3. The query is validated by checking that all attribute names and relation names are valid
 4. An intermediate internal representation for the query is created (query tree or query graph)
 5. Query execution strategy is developed
 6. Query optimizer produces an execution plan
 7. Code generator generates the object code
 8. Runtime database processor executes the code
- Query processing and query optimization

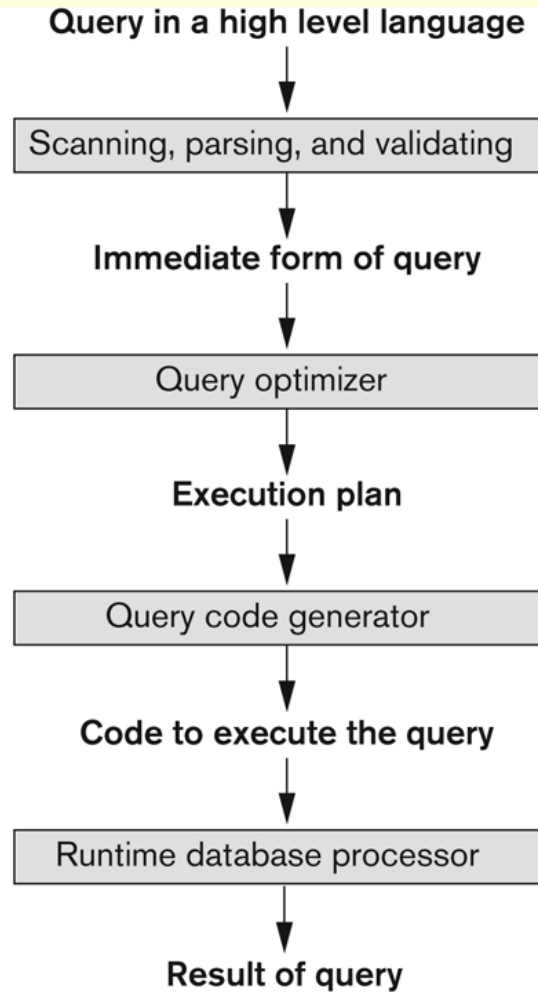
Processing a Query

- Typical steps in processing a high-level query
 1. *Query in a high-level query language like SQL*
 2. **Scanning, parsing, and validation**
 3. *Intermediate-form of query like query tree*
 4. **Query optimizer**
 5. *Execution plan*
 6. **Query code generator**
 7. *Object-code for the query*
 8. **Run-time database processor**
 9. *Results of query*

Scanning , Parsing , Validating

- **Scanner:** The scanner identifies the language tokens such as SQL Keywords, attribute names, and relation names in the text of the query.
- **Parser:** The parser checks the query syntax to determine whether it is formulated according to the syntax rules of the query language.
- **Validation:** The query must be validated by checking that all attributes and relation names are valid and semantically meaningful names in the schema of the particular database being queried.

- **Query Optimization:** The process of choosing a suitable execution strategy for processing a query. This module has the task of producing an execution plan.
- **Query Code Generator:** It generates the code to execute the plan.
- **Runtime Database Processor:** It has the task of running the query code whether in compiled or interpreted mode. If a runtime error results an error message is generated by the runtime database processor.



Code can be:

Executed directly (interpreted mode)
Stored and executed later whenever
needed (compiled mode)

Figure 15.1
Typical steps when
processing a high-
level query.

Translating SQL Queries into Relational Algebra

SQL Queries and Relational Algebra

- SQL query is translated into an equivalent *extended* relational algebra expression --- represented as a query tree
- In order to transform a given query into a query tree, the query is decomposed into *query blocks*
 - **Query block:**
 - The basic unit that can be translated into the algebraic operators and optimized.
 - A query block contains a single SELECT-FROM-WHERE expression, as well as GROUP BY and HAVING clause if these are part of the block.
- The query optimizer chooses an execution plan for each block

COMPANY Relational Database Schema (1)

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

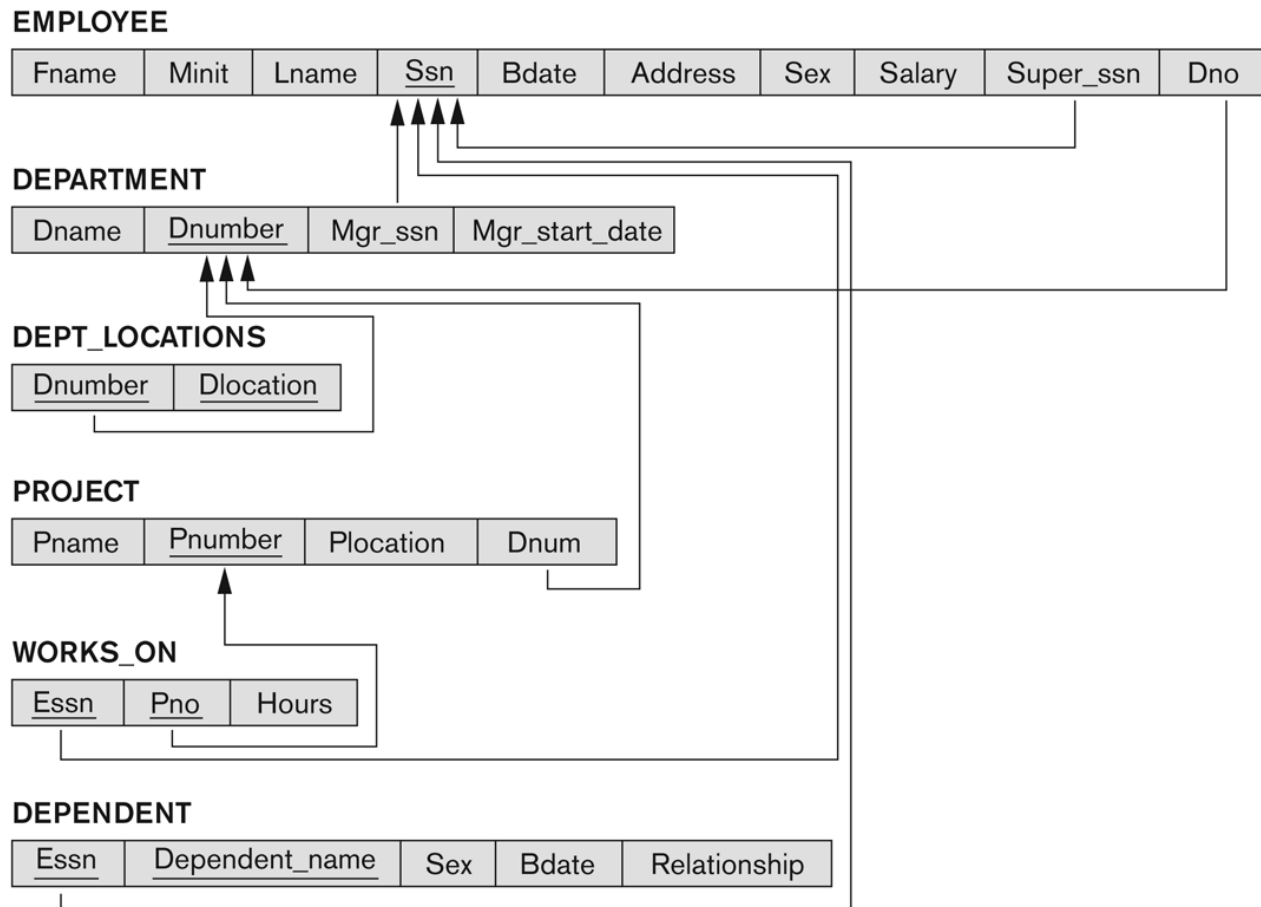
<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------

Figure 5.5
Schema diagram for
the COMPANY
relational database
schema.

COMPANY Relational Database Schema (2)

Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.



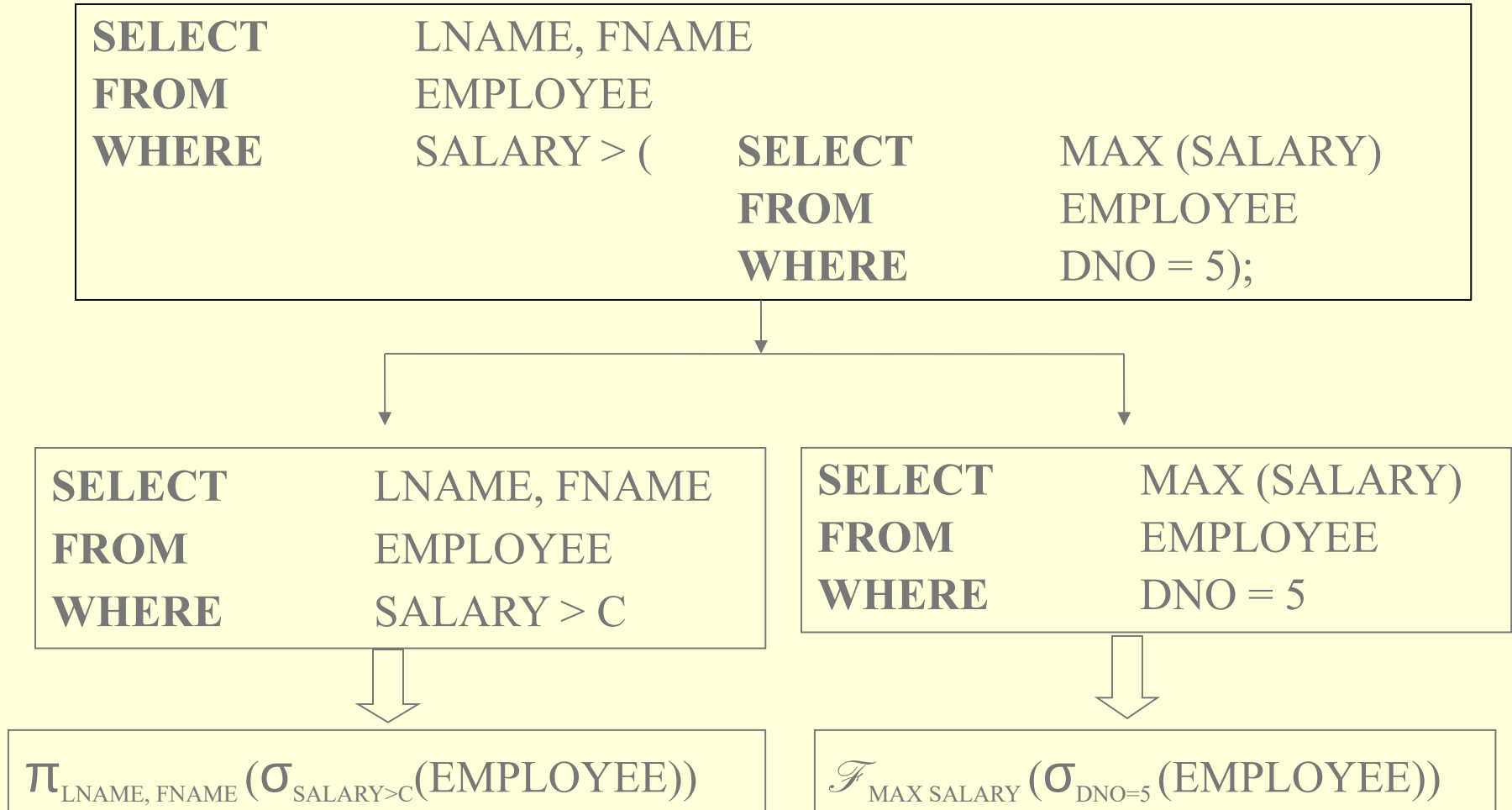
SQL Queries and Relational Algebra (1)

- Example

```
SELECT  Lname, Fname
FROM EMPLOYEE
WHERE Salary > ( SELECT MAX(Salary)
                  FROM EMPLOYEE
                  WHERE Dno = 5      )
```

- Inner block and outer block

Translating SQL Queries into Relational Algebra



SQL Queries and Relational Algebra (2)

- *Uncorrelated nested queries Vs Correlated nested queries*
- A subquery can contain a reference to an object defined in a parent statement. This is called an **outer reference**. A subquery that contains an outer reference is called a **correlated subquery**.
 - `SELECT Name, Description FROM Products WHERE Quantity < 2 *
(SELECT AVG(Quantity) FROM SalesOrderItems WHERE
Products.ID=SalesOrderItems.ProductID);`
- A subquery that does not contain references to objects in a parent statement is called an **uncorrelated subquery**.
 - `SELECT Name, Description FROM Products WHERE Quantity < 2 *
(SELECT AVG(Quantity) FROM SalesOrderItems);`

Implementing basic query operations

Implementing Basic Query Operations

- An RDBMS must provide implementation(s) for all the required operations including relational operators and more
- External sorting
 - Sort-merge strategy
 - Sorting phase
 - Number of file blocks (b)
 - Number of available buffers (n_B)
 - Runs --- (b / n_B)
 - Merging phase --- passes
 - Degree of merging --- the number of runs that are merged together in each pass

Algorithms for External Sorting (1)

- **External sorting:**
 - Refers to sorting algorithms that are suitable for large files of records stored on disk that do not fit entirely in main memory, such as most database files.
- **Sort-Merge strategy:**
 - Starts by sorting small subfiles (**runs**) of the main file and then merges the sorted runs, creating larger sorted subfiles that are merged in turn.

Algorithms for External Sorting (2)

```
set   $i \leftarrow 1$ ;  
      $j \leftarrow b$ ;           {size of the file in blocks}  
      $k \leftarrow n_B$ ;        {size of buffer in blocks}  
      $m \leftarrow \lceil (j/k) \rceil$ ;
```

{Sort Phase}

while ($i \leq m$)

do {

read next k blocks of the file into the buffer or if there are less than k blocks
 remaining, then read in the remaining blocks;
 sort the records in the buffer and write as a temporary subfile;
 $i \leftarrow i + 1$;

}

{Merge Phase: merge subfiles until only 1 remains}

set $i \leftarrow 1$;

$p \leftarrow \lceil \log_{k-1} m \rceil$; { p is the number of passes for the merging phase}

$j \leftarrow m$;

while ($i \leq p$)

do {

$n \leftarrow 1$;

$q \leftarrow \lceil (j/(k-1)) \rceil$; {number of subfiles to write in this pass}

while ($n \leq q$)

do {

read next $k-1$ subfiles or remaining subfiles (from previous pass)
 one block at a time;

merge and write as new subfile one block at a time;

$n \leftarrow n + 1$;

}

$j \leftarrow q$;

$i \leftarrow i + 1$;

}

Figure 15.2

Outline of the sort-
merge algorithm for
external sorting.

Algorithms for External Sorting (3)

- Analysis

Number of file blocks = b

Number of initial runs = n_R

Available buffer space = n_B

Sorting phase: $n_R = \lceil (b/n_B) \rceil$

Degree of merging: $d_M = \text{Min}(n_B - 1, n_R);$

Number of passes: $n_P = \lceil (\log_{d_M}(n_R)) \rceil$

Number of block accesses: $(2 * b) + (2 * b * (\log_{d_M}(n_R)))$

- Example done in the class

Example

- Each tuple is 40 bytes long, 100 tuples per page, 1000 pages.
- Assume we have 5 pages in our buffer pool!

Heuristic-Based Query Optimization

Using Heuristics in Query Optimization(1)

- Process for heuristics optimization
 1. The parser of a high-level query generates an initial internal representation;
 2. Apply heuristics rules to optimize the internal representation.
 3. A query execution plan is generated to execute groups of operations based on the access paths available on the files involved in the query.
- The main heuristic is to apply first the operations that reduce the size of intermediate results.
 - E.g., Apply SELECT and PROJECT operations before applying the JOIN or other binary operations.

Using Heuristics in Query Optimization (2)

- **Query tree:**
 - A tree data structure that corresponds to a relational algebra expression. It represents the input relations of the query as **leaf nodes** of the **tree**, and represents the relational algebra operations as internal nodes.
- An execution of the query tree consists of executing an internal node operation whenever its operands are available and then replacing that internal node by the relation that results from executing the operation.
- **Query graph:**
 - A graph data structure that corresponds to a relational calculus expression. It does *not* indicate an order on which operations to perform first. There is only a *single* graph corresponding to each query.

Using Heuristics in Query Optimization (4)

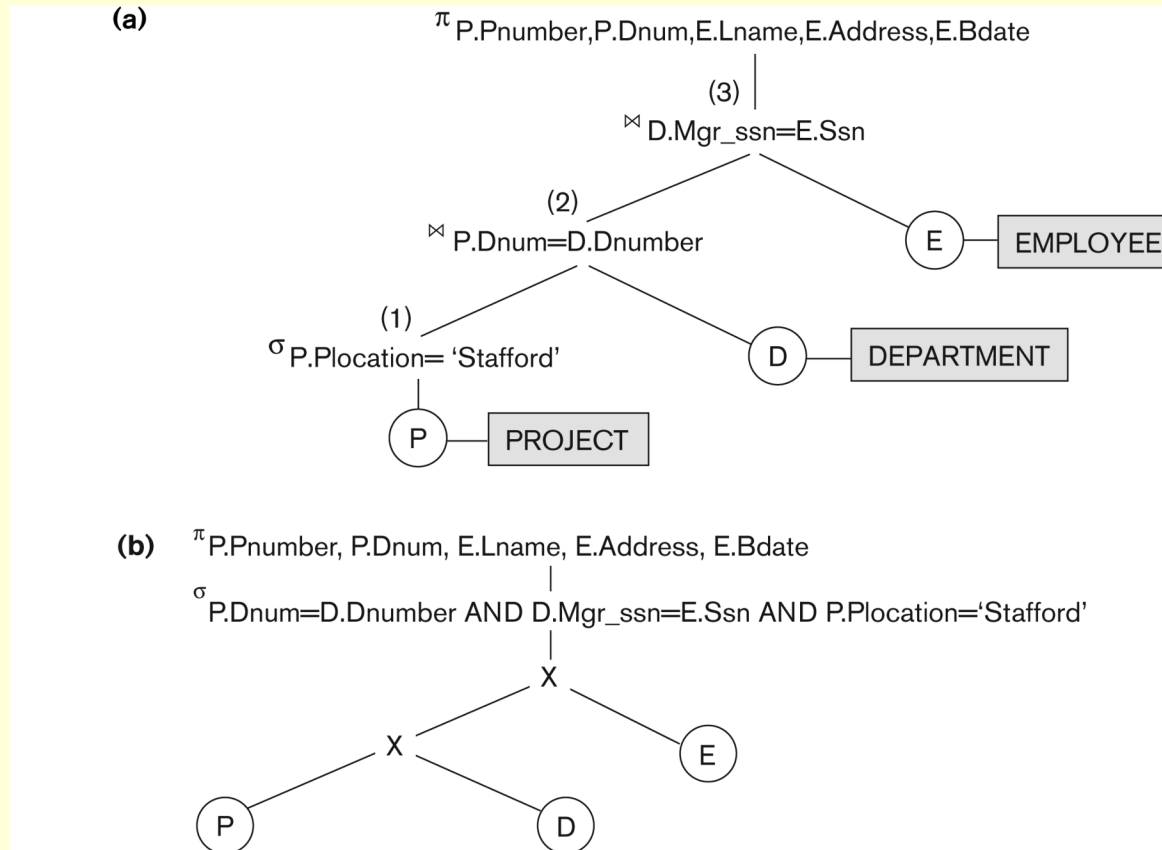


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Using Heuristics in Query Optimization (5)

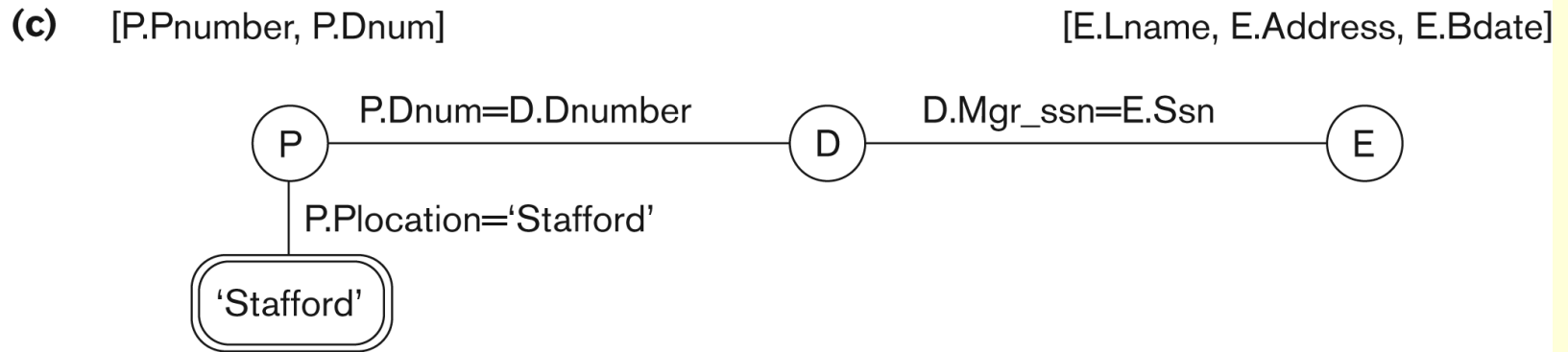


Figure 15.4

Two query trees for the query Q2. (a) Query tree corresponding to the relational algebra expression for Q2. (b) Initial (canonical) query tree for SQL query Q2. (c) Query graph for Q2.

Using Heuristics in Query Optimization (6)

- Heuristic Optimization of Query Trees:
 - The same query could correspond to many different relational algebra expressions — and hence many different query trees.
 - The task of heuristic optimization of query trees is to find a **final query tree** that is efficient to execute.

- Example:

```
Q:      SELECT      LNAME
        FROM        EMPLOYEE, WORKS_ON, PROJECT
        WHERE       PNAME = 'AQUARIUS' AND
        PNMUBER=PNO AND ESSN=SSN          AND
        BDATE > '1957-12-31';
```

Using Heuristics in Query Optimization (7)

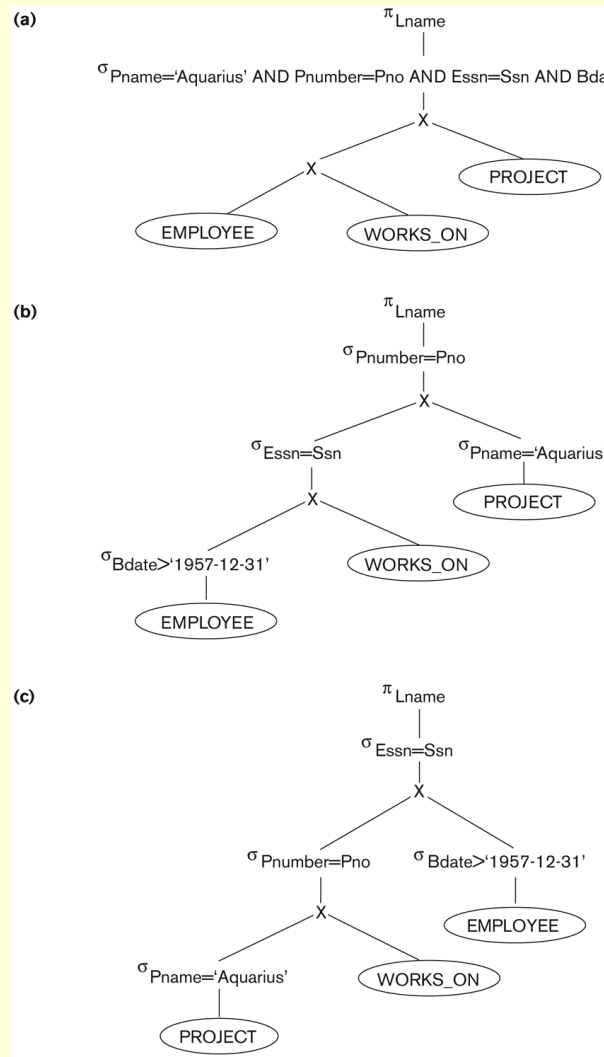


Figure 15.5

Steps in converting a query tree during heuristic optimization.

(a) Initial (canonical) query tree for SQL query Q.

(b) Moving SELECT operations down the query tree.

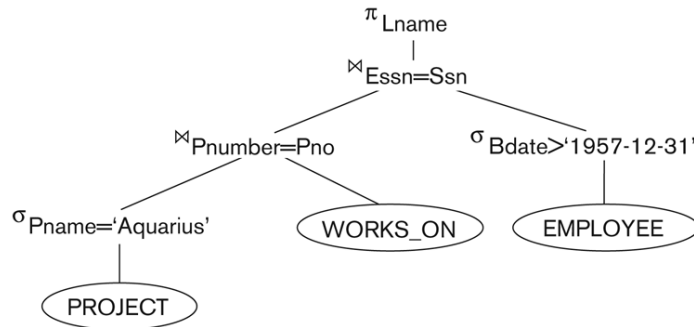
(c) Applying the more restrictive SELECT operation first.

(d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.

(e) Moving PROJECT operations down the query tree.

Using Heuristics in Query Optimization (8)

(d)



(e)

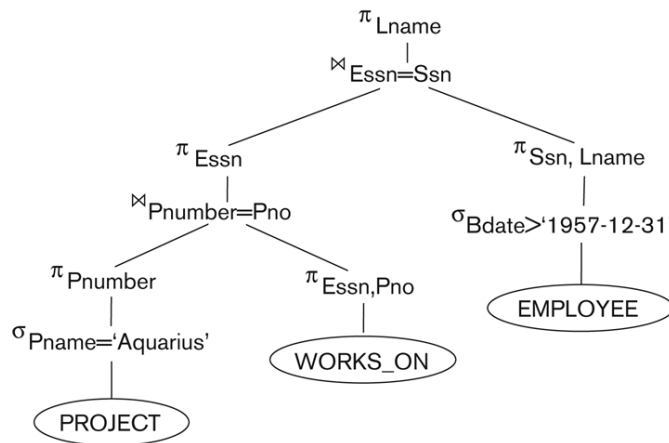


Figure 15.5

Steps in converting a query tree during heuristic optimization.

- (a) Initial (canonical) query tree for SQL query Q.
- (b) Moving SELECT operations down the query tree.
- (c) Applying the more restrictive SELECT operation first.
- (d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.
- (e) Moving PROJECT operations down the query tree.

Using Heuristics in Query Optimization (9)

- General Transformation Rules for Relational Algebra Operations:
 1. Cascade of σ : A conjunctive selection condition can be broken up into a cascade (sequence) of individual σ operations:
 - $\sigma_{c1 \text{ AND } c2 \text{ AND } \dots \text{ AND } cn}(R) = \sigma_{c1} (\sigma_{c2} (\dots (\sigma_{cn}(R)) \dots))$
 2. Commutativity of σ : The σ operation is commutative:
 - $\sigma_{c1} (\sigma_{c2}(R)) = \sigma_{c2} (\sigma_{c1}(R))$
 3. Cascade of π : In a cascade (sequence) of π operations, all but the last one can be ignored:
 - $\pi_{List1} (\pi_{List2} (\dots (\pi_{Listn}(R)) \dots)) = \pi_{List1}(R)$
 4. Commuting σ with π : If the selection condition c involves only the attributes $A1, \dots, An$ in the projection list, the two operations can be commuted:
 - $\pi_{A1, A2, \dots, An} (\sigma_c (R)) = \sigma_c (\pi_{A1, A2, \dots, An} (R))$

Using Heuristics in Query Optimization (10)

- General Transformation Rules for Relational Algebra Operations (contd.):
5. Commutativity of $\bowtie$ (and $\times$): The $\bowtie$ operation is commutative as is the $\times$ operation:
- $R \bowtie_c S = S \bowtie_c R$; $R \times S = S \times R$
6. Commuting σ with $\bowtie$ (or $\times$): If all the attributes in the selection condition c involve only the attributes of one of the relations being joined—say, R —the two operations can be commuted as follows:
- $\sigma_c (R \bowtie S) = (\sigma_c (R)) \bowtie S$
 - Alternatively, if the selection condition c can be written as (c_1 and c_2), where condition c_1 involves only the attributes of R and condition c_2 involves only the attributes of S , the operations commute as follows:
 - $\sigma_c (R \bowtie S) = (\sigma_{c_1} (R)) \bowtie (\sigma_{c_2} (S))$

Using Heuristics in Query Optimization (11)

- General Transformation Rules for Relational Algebra Operations (contd.):
7. Commuting π with $\bowtie$ (or $\times$): Suppose that the projection list is $L = \{A_1, \dots, A_n, B_1, \dots, B_m\}$, where $A_1, \dots, A_n$ are attributes of R and $B_1, \dots, B_m$ are attributes of S . If the join condition c involves only attributes in L , the two operations can be commuted as follows:
- $\pi_L (R \bowtie_C S) = (\pi_{A_1, \dots, A_n} (R)) \bowtie_C (\pi_{B_1, \dots, B_m} (S))$
 - If the join condition C contains additional attributes not in L , these must be added to the projection list, and a final π operation is needed.

Using Heuristics in Query Optimization (12)

- General Transformation Rules for Relational Algebra Operations (contd.):
8. Commutativity of set operations: The set operations $\cup$ and $\cap$ are commutative but “ $-$ ” is not.
 9. Associativity of $\bowtie$, $\times$, $\cup$, and $\cap$: These four operations are individually associative; that is, if θ stands for any one of these four operations (throughout the expression), we have
 - $(R \theta S) \theta T = R \theta (S \theta T)$
 10. Commuting σ with set operations: The σ operation commutes with $\cup$, $\cap$, and $-$. If θ stands for any one of these three operations, we have
 - $\sigma_c (R \theta S) = (\sigma_c (R)) \theta (\sigma_c (S))$

Using Heuristics in Query Optimization (13)

- General Transformation Rules for Relational Algebra Operations (contd.):

- The π operation commutes with $\cup$.

π_L

$$(R \cup S) = (\pi_L(R)) \cup (\pi_L(S))$$

- Converting a (σ, x) sequence into $\bowtie$: If the condition c of a σ that follows a x Corresponds to a join condition, convert the (σ, x) sequence into a $\bowtie$ as follows:

$$(\sigma_c(R \bowtie S)) = (R \bowtie_c S) \quad \bowtie$$

- Other transformations

Using Heuristics in Query Optimization (14)

- Outline of a Heuristic Algebraic Optimization Algorithm:
 1. Using rule 1, break up any select operations with conjunctive conditions into a cascade of select operations.
 2. Using rules 2, 4, 6, and 10 concerning the commutativity of select with other operations, move each select operation as far down the query tree as is permitted by the attributes involved in the select condition.
 3. Using rule 9 concerning associativity of binary operations, rearrange the leaf nodes of the tree so that the leaf node relations with the most restrictive select operations are executed first in the query tree representation.
 4. Using Rule 12, combine a Cartesian product operation with a subsequent select operation in the tree into a join operation.
 5. Using rules 3, 4, 7, and 11 concerning the cascading of project and the commuting of project with other operations, break down and move lists of projection attributes down the tree as far as possible by creating new project operations as needed.
 6. Identify subtrees that represent groups of operations that can be executed by a single algorithm.

Using Heuristics in Query Optimization (15)

- Summary of Heuristics for Algebraic Optimization:
 1. The main heuristic is to apply first the operations that reduce the size of intermediate results.
 2. Perform select operations as early as possible to reduce the number of tuples and perform project operations as early as possible to reduce the number of attributes. (This is done by moving select and project operations as far down the tree as possible.)
 3. The select and join operations that are most restrictive should be executed before other similar operations. (This is done by reordering the leaf nodes of the tree among themselves and adjusting the rest of the tree appropriately.)

Using Heuristics in Query Optimization (16)

- Query Execution Plans
 - An execution plan for a relational algebra query consists of a combination of the relational algebra query tree and information about the access methods to be used for each relation as well as the methods to be used in computing the relational operators stored in the tree.
 - **Materialized evaluation:** the result of an operation is stored as a temporary relation.
 - **Pipelined evaluation:** as the result of an operator is produced, it is forwarded to the next operator in sequence.

8. Using Selectivity and Cost Estimates in Query Optimization (1)

- **Cost-based query optimization:**
 - Estimate and compare the costs of executing a query using different execution strategies and choose the strategy with the lowest cost estimate.
 - (Compare to heuristic query optimization)
- Issues
 - Cost function
 - Number of execution strategies to be considered

Using Selectivity and Cost Estimates in Query Optimization (2)

- Cost Components for Query Execution
 1. Access cost to secondary storage
 2. Storage cost
 3. Computation cost
 4. Memory usage cost
 5. Communication cost
- Note: Different database systems may focus on different cost components.

10. Semantic Query Optimization

- **Semantic Query Optimization:**
 - Uses constraints specified on the database schema in order to modify one query into another query that is more efficient to execute.
- Consider the following SQL query,
SELECT E.LNAME, M.LNAME
FROM EMPLOYEE E M
WHERE E.SUPERSSN=M.SSN AND E.SALARY>M.SALARY
- Explanation:
 - Suppose that we had a constraint on the database schema that stated that no employee can earn more than his or her direct supervisor. If the semantic query optimizer checks for the existence of this constraint, it need not execute the query at all because it knows that the result of the query will be empty. Techniques known as theorem proving can be used for this purpose.